AI-Driven Energy-Efficient Orchestration of User Plane Functions in 5G Core Networks

Shima Afshar Borji[a,*], Roberto Bruschi[a,b], Chiara Lombardo[a,b], Raffaele Bolla[a,b] and Cristina Emilia Costa[b]

[a] DITEN – University of Genoa, Genoa, Italy
[b] CNIT – S2N National Lab, Genoa, Italy

ARTICLE INFO

Keywords:
5G Core Network
User Plane Function
Energy Efficiency
Deep Reinforcement Learning
Network Orchestration

ABSTRACT

The User Plane Function (UPF) is a major contributor to the energy footprint of the 5G Core, yet practical orchestration must balance energy efficiency, Quality of Service (QoS), and actuation stability. We present an AI-driven, closed-loop framework for energy-aware UPF orchestration that selects between a DPDK-based dataplane and a cloud-native user-space dataplane, while supporting horizontal scaling when multiple user-space instances are available. The control problem is formulated as a sequential decision process with an explicit cooldown guard and reconfiguration costs. The controller is trained and evaluated offline in a measurement-grounded digital twin that emulates UPF-attributed power and QoS compliance for each configuration, with the resulting policy intended for subsequent online deployment. The reward balances specific energy consumption, soft QoS penalties, and orchestration costs. On a held-out traffic split, the feed-forward PPO controller reduces mean total attributed energy by 16.9% relative to static DPDK provisioning and by 1.1% relative to a carefully engineered rule-based baseline, without increasing QoS violations, consistently across independent training seeds. Rather than switching aggressively between modes, the learned policy replaces disruptive mode changes with lower-cost horizontal scaling actions. These results support the viability of constraint-aware reinforcement learning as a decision mechanism for energy-efficient dataplane reconfiguration in softwarized 5G Core deployments.

1. Introduction

The fifth generation (5G) of mobile networks has restructured telecommunications infrastructure around a service-based architecture (SBA) in which discrete Network Functions (NFs) communicate through standardized interfaces and can be instantiated, scaled, and terminated independently. Within this architecture, the User Plane Function (UPF) occupies a uniquely demanding position. As the NF responsible for forwarding user traffic, it is one of the most computationally intensive components of the 5G Core (5GC) and a major contributor to its energy footprint [1, 2]. As 5G deployments densify and traffic volumes grow, energy-aware management of the user plane has become an important operational and environmental concern. The information and communication technology sector already accounts for approximately 2–3% of global greenhouse gas emissions [3], and softwarized mobile-core functions contribute to that footprint.

The cloudification of the 5GC has also created a heterogeneous ecosystem of UPF implementations spanning a wide performance–energy spectrum. Kernel-bypass dataplanes can provide high throughput and low forwarding latency, but often sustain a near-constant resource footprint regardless of offered load. Cloud-native user-space implementations typically offer better energy proportionality, but may face throughput limits and QoS degradation under sustained demand. Dynamically choosing between these realization classes, while also selecting the appropriate scale level when multiple instances are available, can reduce user-plane energy consumption without unnecessarily sacrificing service quality. Despite growing interest in UPF placement and routing optimization [2], the problem of runtime switching between heterogeneous UPF realization classes based on their energy–performance trade-offs remains underexplored from a learning perspective.

*Corresponding author
shima.afshar.borji@edu.unige.it (S.A. Borji); roberto.bruschi@unige.it (R. Bruschi); chiara.lombardo@unige.it (C. Lombardo); raffaele.bolla@unige.it (R. Bolla); ccosta@cnit.it (C.E. Costa)
ORCID(s):

Under static provisioning, the most common deployment pattern, a single UPF implementation is kept active regardless of load, either over-provisioning resources to guarantee QoS or under-provisioning and risking Service Level Agreement (SLA) violations at peak demand. Rule-based policies are a natural next step, but they require manual threshold engineering, are brittle near load boundaries, and become increasingly awkward when multiple admissible configurations must be compared under switching delays and cooldown constraints. Practical deployment factors such as reconfiguration cost, delayed actuation, and oscillation risk are also frequently simplified or ignored in theoretical formulations [4, 5]. Deep reinforcement learning (DRL) is therefore attractive not because it is the only possible solution, but because it can optimize sequential decisions directly from interaction data while internalizing delayed consequences and explicit operational constraints [6, 7].

In this paper, we formulate UPF orchestration as a constrained sequential decision problem and solve it with a Proximal Policy Optimization (PPO) controller trained in a data-driven digital twin. The target use case is an online closed-loop orchestrator aligned with standard 5G/NFV functions, including NWDAF, MDAF, and a virtualized orchestration layer, but the present study validates the decision logic offline in a measurement-grounded twin rather than through live closed-loop deployment on the testbed. By integrating network-level analytics (NWDAF), infrastructure-level telemetry (MDAF/IDAF), and a learning-based decision module, the framework instantiates the broader vision of intelligence-driven service orchestration in next-generation mobile networks. The controller reasons over traffic context, a scalar QoS-compliance signal, and an attributed power signal derived from profiling measurements. The environment also embeds a cooldown guard and asymmetric reconfiguration costs so that actuation stability is modeled within the decision process.

Contributions. The main contributions of this paper are as follows:

1. **Constraint-aware orchestration formulation.** We define a UPF orchestration problem over heterogeneous realization modes and scale levels, and model cooldown and reconfiguration costs explicitly in the control loop and reward.
2. **Measurement-grounded digital twin.** We profile a DPDK-based SD-Core UPF and a cloud-native USR UPF on a physical testbed, fit per-mode profiling models for attributed power and QoS compliance, and use them to build a deterministic digital twin for safe offline policy training.
3. **Comparative evaluation of PPO-based control.** We compare feed-forward and recurrent PPO controllers against static and rule-based baselines on two distinct real mobile-traffic traces derived from the same measurement campaign. The results show that the feed-forward PPO controller improves attributed energy over the baselines while maintaining the same QoS violation rate as the rule-based selector under identical guard semantics. Cross-trace generalization is validated on a second workload, and statistical robustness is assessed through Wilcoxon signed-rank tests.

The remainder of the paper is organized as follows. Section 2 reviews the relevant literature. Sections 3 and 4 present the system model and learning formulation. Sections 5 and 6 describe the testbed, profiling procedure, digital twin, and experimental protocol. Section 7 reports the results, including training convergence diagnostics, cross-trace generalization, and statistical significance analysis. Section 8 concludes the paper.

2. Background & Related Work

2.1. Energy Footprint of the 5G User Plane

The energy consumption of 5G infrastructure has grown alongside deployment scale, with the information and communication technology sector already responsible for approximately 2–3% of global greenhouse gas emissions [3]. Within the 5GC, the UPF is a major consumer at the per-NF level. Bellin et al. [8, 1] profile open-source 5GC stacks across bare-metal, container, and VM deployments and show that the virtualization layer and the software implementation materially affect the resulting energy footprint, with the UPF contributing a large share of the per-NF power draw. These results motivate energy-aware user-plane management rather than assuming that softwarization alone improves efficiency.

2.2. UPF Implementation Diversity and Energy–Performance Trade-offs

Dynamic UPF selection is meaningful only if the available implementations expose materially different energy–performance behavior. Recent measurement studies confirm this. Christakis et al. [9] evaluate multiple UPF implementations in a real-world OAI-based 5G setup and show that DPDK-based VPP can keep CPUs busy even when idle

because of continuous polling, whereas hardware-offloaded datapaths shift the processing burden elsewhere. Rischke et al. [10] compare tail-latency behavior across acceleration technologies and show that DPDK can sustain markedly lower tail latency than user-space alternatives under demanding traffic. On the energy side, Bolla et al. [11] report different low-load and high-load power trends for VPP and eBPF/XDP. These studies do not imply that one realization dominates across all operating conditions; rather, they show that implementation choice changes the attainable energy–QoS trade-off.

2.3. Energy-Aware UPF and VNF Orchestration

A common response to the above trade-off is threshold-based control. Our prior work [12] explored binary DPDK/USR switching with rule-based logic and showed that dynamic selection can reduce energy relative to static DPDK provisioning. More broadly, Chen et al. [2] formulate UPF placement and traffic routing as a mixed-integer program, demonstrating that deployment choices materially affect 5GC energy expenditure. Related work on service-function chaining and VNF placement also addresses joint energy and traffic cost optimization through integer programming and heuristics [13, 14]. These approaches are valuable, but they target static or slowly updated placement decisions rather than runtime switching among heterogeneous realizations.

Threshold-based autoscaling has also been studied analytically. Tournaire et al. [5] model hysteresis queues and show that dead-band design is important for suppressing oscillation. Soto et al. [4] compare DRL, PID control, and threshold-based autoscaling for VNFs, finding that threshold logic can be outperformed in steady-state efficiency and often needs additional oscillation-control mechanisms. In our setting, the limitation is not that thresholds are always poor, but that they become harder to engineer and less expressive as the controller must compare multiple configurations under shared cooldown constraints. Gausseran et al. [15] further address reconfiguration continuity through make-before-break mechanisms, but not the control logic that decides when and how to reconfigure. Leyva-Pupo and Cervelló-Pastor [**?**] use supervised ML to predict QoS degradation and schedule UPF placement reconfigurations proactively, reducing unnecessary reconfiguration events by over 50% compared to periodic and threshold-based baselines. Their work focuses on mobility-driven placement updates rather than runtime switching between heterogeneous UPF realization classes.

2.4. Deep Reinforcement Learning for VNF and Network Resource Management

DRL addresses some of the above limitations by optimizing a sequential control policy from interaction data. Pei et al. [6] apply Double DQN to VNF placement in SDN/NFV environments and report near-optimal policies under resource constraints. Solozabal et al. [16] use policy-gradient methods for constrained VNF placement. These works do not model cooldown guards or switching costs, and they focus on placement rather than time-series control. PPO [17] has become a common choice in networking because its clipped surrogate objective tends to stabilize policy updates in non-stationary reward settings. Chen et al. [18], Kuai et al. [19], and Santos et al. [20] report encouraging results for PPO-based communications and VNF orchestration tasks. Bellin et al. [**?**] apply PPO to energy-aware UPF selection in a distributed edge-cloud deployment, using real-time power consumption metrics to guide session-level assignment across heterogeneous hosts. In our case, the action space is small ($|\mathcal{C}| = 3$), so PPO serves as a stable policy-gradient baseline for a sequential control problem with delayed consequences induced by cooldown and switching costs.

2.5. Digital Twins for Safe RL Policy Training

Training RL agents directly on operational networks is impractical because exploratory reconfigurations carry service risk. Data-driven digital twins provide a controlled alternative by approximating the environment offline and using that approximation for policy learning. Tao et al. [21] use a digital twin to pre-train a DRL policy for sliced-network admission control, while Wang et al. [22] propose a GNN-based twin for network slicing management. These works support the use of offline profiling models for safe policy development. Our work adapts that paradigm to UPF orchestration using profiling models fitted to profiling measurements collected on a physical testbed.

2.6. Positioning of This Work

The existing literature shows that different UPF realizations exhibit distinct energy–performance trade-offs, that threshold-based control becomes increasingly difficult to design as the configuration space grows, and that digital twins provide a practical way to train RL policies safely. However, measurement-grounded approaches to UPF orchestration that combine these aspects in a single framework remain limited.

In particular, the joint treatment of heterogeneous realization modes, explicit cooldown constraints, asymmetric reconfiguration costs, and horizontal scaling within a unified control formulation has received relatively little attention.

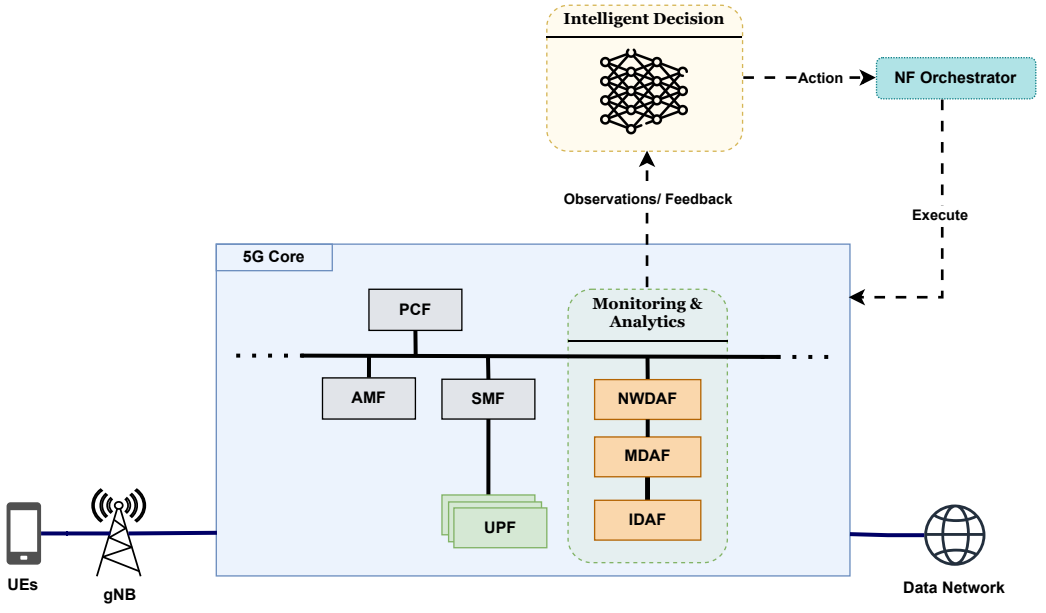


Figure 1: Target online closed-loop UPF orchestration architecture. The present paper validates the decision logic offline in a measurement-grounded digital twin before any live deployment.

This work addresses this gap by combining a physically profiled digital twin with a constraint-aware PPO controller and a systematic comparison against static and rule-based baselines.

3. System Architecture

The proposed framework is organized as a closed control loop in which four functional blocks interact continuously, as illustrated in Figure 1. The 5GC exposes user-plane service through the UPF, while a monitoring and analytics block extracts slot-level indicators from network and infrastructure telemetry. These indicators are consumed by a decision module that produces a configuration command. The command is enforced by an orchestration layer that applies the corresponding actuation to the UPF deployment under operational guards that limit unstable behavior. Figure 1 therefore represents the intended online deployment architecture; in the present paper, the decision logic mapped to this architecture is trained and evaluated offline in the digital twin described later.

3.1. 5GC User-Plane Model

The 5GC domain includes the control-plane functions and the user-plane function. In the proposed architecture, the UPF is the controlled element subject to orchestration. Control-plane functions are treated as external to the optimization loop; they provide service context and support UPF endpoint configuration, but they are not decision variables in our model.

3.2. UPF Configuration Space

We consider two UPF realization classes with different performance–energy characteristics:

DPDK-based UPF is a kernel-bypass, polling-driven packet-processing pipeline designed for high throughput and low latency.

User-space UPF is a general-purpose software UPF deployed as containerized instances that can be scaled horizontally.

We model the deployment as a discrete configuration that captures both the realization mode and its horizontal scale. Let $\mathcal{M}$ be the set of supported UPF modes. In this work, $\mathcal{M} = \{\text{DPDK}, \text{USR}\}$, where DPDK denotes a DPDK-based dataplane and USR denotes a cloud-native user-space dataplane. For each mode $m \in \mathcal{M}$, we define a finite set

of admissible instance counts $\mathcal{K}_m \subset \mathbb{N}$. A configuration at slot t is represented as

$$c_t = (m_t, k_t), \qquad m_t \in \mathcal{M},\ k_t \in \mathcal{K}_{m_t}, \tag{1}$$

and the overall configuration space is

$$\mathcal{C} = \bigcup_{m \in \mathcal{M}} \{m\} \times \mathcal{K}_m. \tag{2}$$

The concrete mode set $\mathcal{M}$, admissible instance counts $\mathcal{K}_m$, and resulting configuration space $\mathcal{C}$ used in the evaluation are specified in Section 5.

3.3. Monitoring and Analytics

Monitoring and analytics operate on a slotted time base. Time is divided into control slots of duration Δ, indexed by $t \in \mathbb{N}$. During each slot, raw telemetry is collected from network functions and the hosting infrastructure, then aggregated into slot-level indicators aligned to the slot timestamp. We collect these indicators in an observation vector x_t, which is used by the decision module.

The indicators fall into three groups. Traffic indicators summarize the offered load during the slot. QoS indicators summarize service compliance during the slot. Resource and energy indicators summarize the footprint of the active deployment. In the remainder of the paper, the energy-related signal is represented through slot-averaged UPF-attributed power P_t on the hosting node; slot energy is then derived as $E_t = P_t\Delta$ when needed. The emphasis on attributed power is important: the quantities used in the controller and evaluation are process-level estimates attributed to the UPF processes rather than full server power measurements.

The required indicators can be obtained through analytics functions supported by the 5G SBA. Network-facing analytics can be provided through the Network Data Analytics Function (NWDAF), which collects and exposes network-level indicators to consumers [23]. Infrastructure-facing indicators, including resource usage and power-related metrics, can be exposed through a Management Data Analytics Function (MDAF). In energy-oriented realizations, the MDAF can retrieve such KPIs from cloud-native infrastructure resources through the metrics exposed by an infrastructure data collection function (IDAF) rather than by directly accessing the infrastructure [24].

3.4. Orchestration and Actuation Guard

At each control slot, the orchestration layer receives a configuration command and translates it into an actuation on the UPF deployment. Actuation updates the active realization mode and, when applicable, the number of active instances. The resulting deployed state for slot t is represented by the executed configuration $c_t \in \mathcal{C}$.

To prevent rapid oscillations, configuration changes are regulated by an explicit cooldown guard. Let u_t be the number of elapsed control slots since the last executed reconfiguration, and let I_c be the cooldown length in slots. Given a requested configuration command $a_t \in \mathcal{C}$, the executed configuration is obtained as

$$c_t = \begin{cases} a_t, & \text{if } u_t \geq I_c, \\ c_{t-1}, & \text{otherwise.} \end{cases} \tag{3}$$

The cooldown state is reset whenever a reconfiguration is executed and increases by one slot otherwise. This guard is applied uniformly across all controllers considered in the paper. Embedding it in the environment transition function makes the minimum dwell time part of the control problem itself rather than a post-processing rule.

3.5. Decision Interface

At slot t, the decision module receives the observation vector x_t together with the current deployment context and outputs a configuration command $a_t \in \mathcal{C}$. The orchestration layer enforces the actuation guard in Section 3.4 to obtain the executed configuration $c_t \in \mathcal{C}$, which determines the UPF deployment during slot t and influences the observations collected at subsequent decision boundaries.

4. RL-Based Orchestration Framework

This section formalizes the decision module as a sequential decision-making problem and describes the learning environment used in the evaluation. The formulation follows the same control period Δ and the guarded actuation semantics introduced in Section 3.5 and Section 3.4. In the present study, all controller learning and comparison are carried out offline in the digital twin rather than by executing the controller live on the physical testbed.

4.1. Decision Process Formulation

We formulate the closed-loop interaction as a Markov decision process defined at decision epochs indexed by $t \in \mathbb{N}$. At each epoch, the agent observes x_t and issues a configuration command $a_t \in \mathcal{C}$. The orchestration layer enforces the cooldown guard and determines the executed configuration. A reward r_t is then computed from the power and QoS outcomes observed over the subsequent control interval, and the next observation x_{t+1} becomes available at the following decision boundary.

The learning objective is to find a policy π_θ that maximizes the expected discounted return

$$J(\pi_\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \tag{4}$$

where $\gamma \in (0, 1]$ is the discount factor and T is the episode length.

In practice, the agent observes aggregated indicators rather than the full latent network state. The problem is therefore only partially observed from the controller's perspective. We address this by considering multiple observation constructions, including finite traffic history, optional forecast inputs, and recurrent policies. The concrete observation schema used in each experiment is specified in Section 6.

4.2. Digital-Twin Environment and Outcome Signals

Direct exploration on an operational 5GC user plane is impractical because disruptive reconfigurations carry real service cost. We therefore employ a data-driven digital twin that provides a controlled environment for learning and for fair comparison across controllers. The twin emulates the outcome of applying an executed configuration c_t under the traffic conditions of the corresponding control interval.

For each decision epoch t, the twin returns two outcome signals. The first is the UPF-attributed power draw P_t under c_t, represented as the interval-averaged attributed power over the control period. The second is a QoS outcome Q_t that summarizes service compliance for the same period through a scalar performance indicator. These signals are computed with the same procedure for all evaluated controllers and are used to construct the reward in Section 4.5 and the evaluation metrics in Section 7.

4.3. Observation Space and Schemas

The agent input x_t is constructed from the indicator families introduced in Section 3.3. It summarizes traffic conditions, service outcomes, and the energy-related signal observed at the decision boundary. In addition, x_t includes the minimal deployment context required by the actuation semantics, namely the previously executed configuration and the cooldown status.

We evaluate four observation schemas that differ only in how traffic context is provided:

- **Instantaneous:** x_t includes the current traffic load ρ_t together with the current QoS and energy indicators and the deployment context.
- **History-augmented:** x_t augments the instantaneous schema with a window of past traffic loads $\rho_{t-W+1:t}$ of length W.
- **Forecast-augmented:** x_t augments the instantaneous schema with a forecast vector $\hat{\rho}_{t+1:t+H}$ of horizon H.
- **Hybrid:** x_t includes both the history window $\rho_{t-W+1:t}$ and the forecast vector $\hat{\rho}_{t+1:t+H}$ in addition to the instantaneous indicators.

These schemas address the fact that the controller observes aggregated indicators rather than the full latent network state. For each experiment, the schema and its parameters (W, H) are fixed and kept identical across the compared controllers.

4.4. Action Space and Configuration Encoding

The decision module selects configuration commands from the configuration space $\mathcal{C}$ defined in Section 3.2. In the implementation, commands are represented as discrete action indices. A one-to-one mapping is used between the finite set of admissible configurations and the discrete action set, so that each action index corresponds to exactly one configuration command.

4.5. Reward Design

The reward balances energy efficiency, QoS compliance, and orchestration stability. Energy efficiency is captured through the specific energy consumption

$$\mathrm{SEC}_t = \frac{P_t}{\max(\rho_t, \epsilon)}, \tag{5}$$

where P_t is the interval-averaged attributed power returned by the digital twin, ρ_t is the offered load over the same interval, and ϵ is a small constant used for numerical stability.

QoS compliance is enforced through a soft penalty based on the scalar compliance indicator Q_t and a target threshold τ:

$$\mathcal{L}_t^{\mathrm{QoS}} = \lambda_{\mathrm{QoS}} \max(0, \tau - Q_t), \tag{6}$$

where λ_{QoS} controls the penalty strength.

Orchestration stability is encouraged by penalizing executed reconfigurations. Let c_{t-1} and c_t be the executed configurations before and after the actuation guard. The orchestration cost distinguishes three event types:

$$\mathcal{L}_t^{\mathrm{ORCH}} = c_{\mathrm{DPDK}}\, \mathbb{I}\{\text{switch to DPDK}\} \;+\; c_{\mathrm{USR}} \cdot n_t^{+} \;+\; c_{\mathrm{dn}}\, \mathbb{I}\{\text{scale-down within USR}\}, \tag{7}$$

where n_t^{+} is the number of USR instances launched at epoch t (equal to k_t on a mode switch to USR, or to the number of added replicas on a scale-up within USR mode, and zero otherwise). Scale-down is penalized as a fixed event because it always removes exactly one instance in the current configuration space.

The cost c_{DPDK} reflects the overhead of activating the DPDK-based UPF, whose startup takes approximately 24 s (mean over 100 repetitions on the physical testbed). The cost c_{USR} is charged per USR instance launched; each container starts in approximately 3.3 s. The scale-down cost c_{dn} is the smallest because draining a replica does not interrupt service on the remaining instance. The concrete values ($c_{\mathrm{DPDK}} = 0.03$, $c_{\mathrm{USR}} = 0.012$, $c_{\mathrm{dn}} = 0.003$) are reported in Table 3.

The resulting reward is

$$r_t = -\alpha\, \mathrm{SEC}_t - \mathcal{L}_t^{\mathrm{QoS}} - \mathcal{L}_t^{\mathrm{ORCH}}, \tag{8}$$

where α scales the SEC contribution. Using SEC rather than raw power in the energy term discourages trivially low-power but low-throughput operating points. The weights and cost parameters used in the experiments are reported in Section 6.

4.6. Learning Algorithm

The policy π_θ is learned using Proximal Policy Optimization (PPO), an on-policy actor–critic method that supports discrete action spaces and delayed-consequence control. PPO learns a stochastic policy over configuration commands together with a value function that estimates the expected return. Training optimizes the objective in (4) from trajectories generated under the guarded actuation semantics defined in Section 3.4.

We consider two PPO parameterizations. In the feed-forward variant, temporal context is supplied through the observation construction. In the recurrent variant, the policy includes an internal memory state that integrates information across decision epochs. We do not claim PPO is the only suitable method for a small configuration space; rather, it provides a consistent policy-gradient baseline for a constrained sequential-control setting and a natural path toward larger future configuration spaces. Training convergence and entropy dynamics for both variants are analyzed in Section 7.7. The specific variant and training hyperparameters used in each experiment are reported in Section 6.

4.7. Baseline Controllers

We compare the PPO-based controller against static and rule-based baselines. These baselines are part of the controller methodology rather than of the workload setup, so we define them here before introducing the evaluation protocol. All controllers operate on the same configuration space $\mathcal{C}$ and, unless explicitly stated otherwise, are evaluated through the same digital twin described in Section 5.3.

Static provisioning. We include static baselines in which the UPF configuration is kept constant over the full evaluation horizon. In the reported results, we consider static DPDK and static USR($k = 1$), which provide references for the case where no dynamic orchestration is performed.

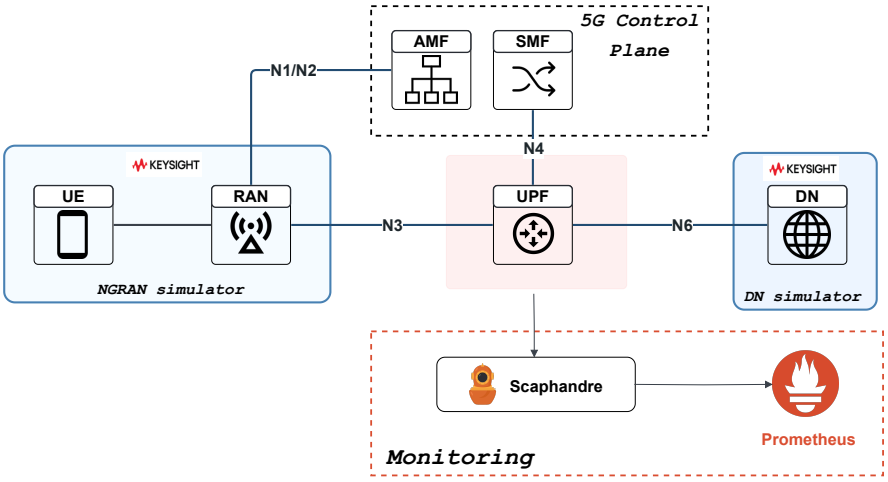


Figure 2: Physical testbed architecture used for UPF profiling and metric collection. The Keysight LoadCore platform simulates UE/RAN (N3) and data-network (N6) endpoints. The 5G control plane (AMF, SMF) manages sessions via N4. Scaphandre monitors power attributed to the UPF processes via RAPL-derived estimates and exports metrics to Prometheus.

Rule-based threshold selector. This baseline follows threshold logic derived from the profiling curves. Offline, we compare the attributed-power and QoS profiling curves of the admissible configurations and identify the throughput regions in which each configuration is preferred under the target QoS threshold. Online, the controller maps the one-step-ahead traffic forecast to the corresponding configuration region and issues the associated command. To reduce oscillations around region boundaries, it applies a dead-band around each threshold and the same cooldown guard used by the PPO controllers.

Offline-optimal reference. We additionally compute a non-causal offline-optimal reference using exact finite-horizon dynamic programming over the deterministic digital twin. This planner has access to the full test-sequence load trajectory and uses the same configuration space, cooldown semantics, reconfiguration costs, and discounted cumulative reward objective as PPO, with $\gamma = 0.995$. It is therefore not a deployable controller, but it provides an upper bound with respect to the modeled control problem and quantifies how much of the available performance gap is captured by the causal controllers.

5. Physical Testbed and Data Collection

This section describes the physical infrastructure used to characterize UPF behavior, the profiling campaign that produced the data used to fit the profiling models, and the measurement assumptions that underpin the digital twin used in the evaluation.

5.1. Infrastructure and UPF Configurations

The testbed consists of a commodity server running both UPF instances and the traffic generator on the same physical host, with power measurements obtained at the process level. The DPDK-based UPF is realized using the SD-Core user plane [**?**], which provides a kernel-bypass, polling-driven packet-processing pipeline. The cloud-native UPF is realized using the Open Air Interface (OAI) UPF [**?**], hereafter referred to as the USR UPF, and is deployed as containerized instances managed by a Kubernetes orchestrator that supports horizontal scaling through replica control. Figure 2 should therefore be read as the profiling and measurement setup used to characterize UPF behavior; the PPO controller is not trained or evaluated in live closed loop on this physical setup in the current paper.

The concrete configuration space instantiated on this testbed is $\mathcal{M} = \{\text{DPDK}, \text{USR}\}$ with $\mathcal{K}_{\text{DPDK}} = \{1\}$ and $\mathcal{K}_{\text{USR}} = \{1, 2\}$, yielding a total of $|\mathcal{C}| = 3$ admissible configurations: one DPDK instance, one USR instance, and two USR instances. A single DPDK instance is sufficient to serve the full load range considered in the evaluation

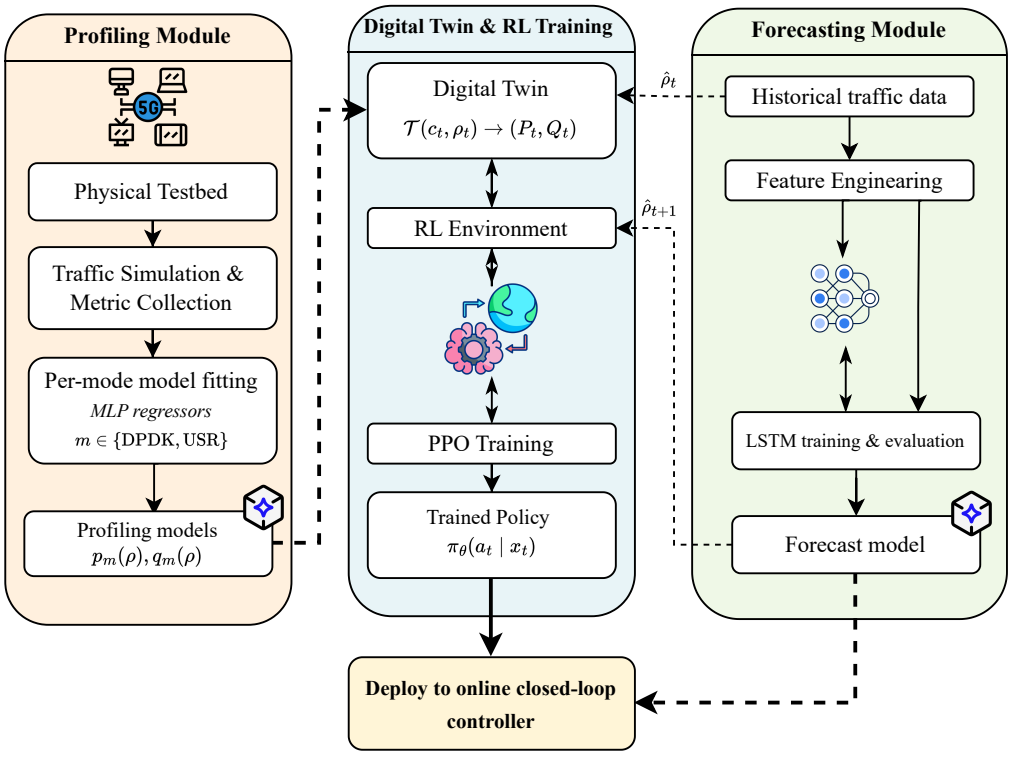


Figure 3: Offline workflow used in this paper for data collection, profiling-model fitting, forecasting, and PPO policy learning. The trained policy π_θ, forecast model, and profiling models (p_m, q_m) are produced offline; the final deployment box indicates intended future use in the online control loop and is not part of the present evaluation.

without performance degradation, so no horizontal scaling is defined for that mode. The USR mode supports up to two instances, reflecting the capacity limit of a single USR instance under the traffic conditions of this work.

Power consumption is measured with Scaphandre, an open-source monitoring tool that estimates per-process energy draw from hardware counters exposed by the RAPL interface. Infrastructure-level metrics including CPU utilization and memory are collected via Prometheus. Traffic is generated using stateless UDP flows with a fixed packet size of 1250 bytes, produced by Keysight LoadCore [**?**].

Because the power signal is process-attributed rather than a full server power trace, the watt and watt-hour quantities reported in this paper should be interpreted as *UPF-attributed estimates* used consistently for profiling, reward construction, and controller comparison within this testbed.

5.2. Profiling Campaign

Figure 3 illustrates the three-stage offline workflow used in this paper. The profiling module characterizes the two UPF realizations on the physical testbed and produces the profiling-model repository, the forecasting module prepares the traffic workload and trains the one-step-ahead predictor, and the digital-twin/RL module assembles these components into a simulation environment in which PPO is trained and evaluated offline. The bottom "Deploy to online closed-loop controller" box represents the intended later deployment target of the trained artifacts and is not part of the experimental validation reported here.

The profiling models are trained from a profiling dataset collected by subjecting each UPF configuration to a sweep of 22 offered-load levels spanning the range from 0.01 Mbps to 5 Gbps. The load levels are defined by the packet-rate sequence at the fixed packet size. Each load level is sustained for a fixed measurement window and repeated across

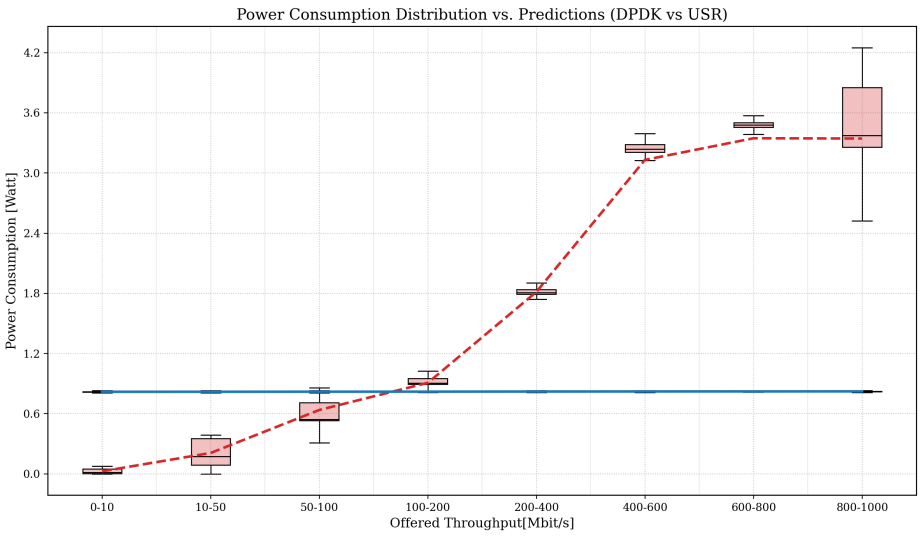


Figure 4: Process-attributed power-consumption distributions and MLP profiling-model predictions for the DPDK (blue, solid) and USR (red, dashed) realizations as a function of offered throughput. Box plots show empirical variability across profiling runs.

multiple runs to capture measurement variability. The traffic direction is downlink only, consistent with the traffic model adopted for the evaluation workload.

QoS is assessed against the service requirements of 5QI=9 (video buffered streaming, non-GBR) as standardized in 3GPP TS 23.501 [25]. The corresponding thresholds are a packet delay budget $D_{\max} = 300$ ms, a jitter bound $J_{\max} = 100$ ms, and a packet error rate $L_{\max} = 10^{-6}$. Each KPI is mapped to a bounded score, and the final scalar QoS-compliance signal used by the twin is defined in Section 5.3.

Figure 4 shows the measured attributed-power distributions and fitted profiling curves for both UPF realizations as a function of offered throughput. In this testbed, the DPDK-based UPF exhibits a nearly load-independent attributed-power profile at approximately 0.75 W, while the USR attributed power increases monotonically from near zero at idle to approximately 3.2 W at saturation. Figure 5 shows the corresponding QoS-compliance scores. DPDK maintains near-perfect compliance ($Q \approx 0.96$) across the full load range. The USR sustains compliance above $\tau = 0.9$ up to approximately 400 Mbps per instance, beyond which the score degrades sharply. These profiles motivate the orchestration problem by showing that the two realizations expose different attributed-power and QoS behavior across load.

5.3. Profiling Models and Digital-Twin Construction

The evaluation relies on a data-driven digital twin that emulates the user-plane outcomes of applying a configuration $c_t = (m_t, k_t) \in C$ under load ρ_t. No controller is executed live on the physical testbed in this study; instead, all results are obtained by replaying the workload through this common profiling-model-based environment. The twin provides two outcome signals for each control interval:

- the interval-averaged UPF-attributed power draw P_t; and
- a scalar QoS-compliance score Q_t for the same interval.

Both signals are obtained from profiling models trained offline on the profiling data. For each UPF realization mode $m \in \mathcal{M}$, we fit two MLP regressors, one for attributed power and one for QoS compliance, using per-instance offered load as input. Denoting these profiling models by $p_m(\rho)$ and $q_m(\rho)$, the twin evaluates

$$(P_t, Q_t) = \mathcal{T}(c_t, \rho_t), \tag{9}$$

where $\mathcal{T}(\cdot)$ applies the mode-specific profiling models and accounts for horizontal scaling when $k_t > 1$.

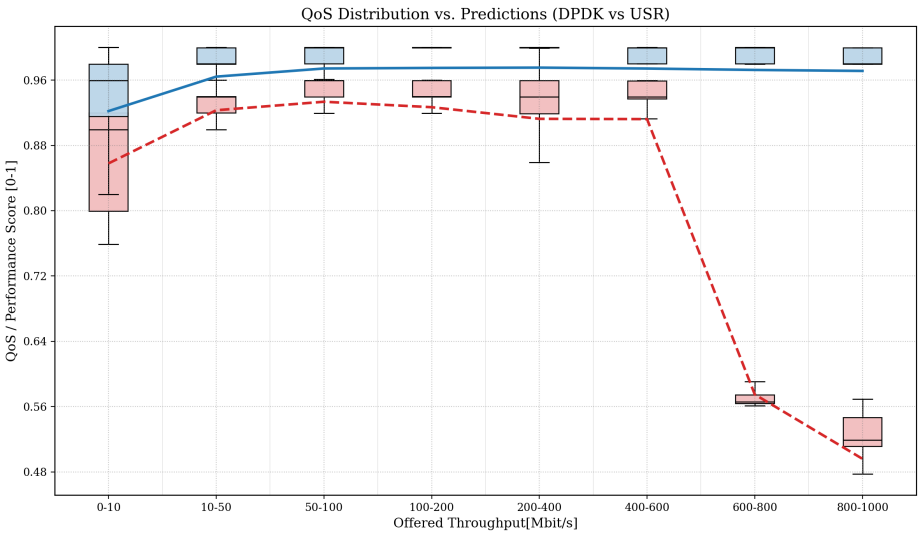


Figure 5: QoS-compliance score distributions and MLP profiling-model predictions for both UPF realizations. The dashed line marks the compliance target $\tau = 0.9$. The USR score degrades sharply above approximately 400 Mbps per instance.

The QoS outcome is represented through a scalar compliance score Q_t obtained by aggregating multiple KPIs observed over the control interval. Let S_t^{lat}, S_t^{jit}, and S_t^{loss} denote bounded component scores associated with delay, jitter, and packet loss, respectively. Each component is mapped to a normalized score through a bounded scoring function with service thresholds $(D_{\max}, J_{\max}, L_{\max})$. The resulting QoS score is computed as

$$Q_t = w_{\text{lat}} S_t^{\text{lat}} + w_{\text{jit}} S_t^{\text{jit}} + w_{\text{loss}} S_t^{\text{loss}}, \tag{10}$$

with non-negative weights $\sum_i w_i = 1$. In the evaluation, QoS components are weighted as $(w_{\text{lat}}, w_{\text{jit}}, w_{\text{loss}}) = (0.40, 0.40, 0.20)$, reflecting equal emphasis on delay and jitter compliance for buffered video-streaming traffic under 5QI=9 and a lower weight on packet loss.

For multi-instance configurations, we assume that traffic is distributed across the k_t active instances by a load-balancing mechanism that yields an approximately even split. Accordingly, the per-instance offered load is approximated as ρ_t/k_t. Under the balanced-split assumption, the twin computes

$$P_t = k_t \, p_{m_t}\left(\frac{\rho_t}{k_t}\right), \tag{11}$$

and derives the interval QoS-compliance score as

$$Q_t = q_{m_t}\left(\frac{\rho_t}{k_t}\right). \tag{12}$$

For each mode and target, a small MLP regressor is trained on an 80/20 held-out split of the profiling data, using log-transformed offered throughput as the sole input feature. Model complexity (depth, regularization) is chosen per target to match the difficulty of the regression; the DPDK attributed-power model is shallow because the target is nearly constant, whereas the USR models use deeper architectures with batch normalization and dropout to capture the steeper load–response curves. Table 1 reports held-out accuracy for the four fitted profiling models.

The profiling models introduce a controlled approximation: the twin captures the mean load–response relationship but does not reproduce measurement noise or transient effects. The low R^2 values for the DPDK models in Table 1 reflect near-zero target variance rather than poor model fit: when the response is approximately constant across the load range, R^2 is uninformative as a goodness-of-fit metric, while the corresponding RMSE of 0.0039 W confirms accurate prediction. This provides a deterministic evaluation environment that removes one source of variance from the controller comparison, while the profiling variability shown in Figures 4 and 5 gives a qualitative indication of the uncertainty present in the underlying system.

Table 1
Held-out accuracy of the profiling models used by the digital twin. Metrics are computed on the 20% held-out split.

Mode	Target	RMSE	R^2
DPDK	QoS score	0.0847	0.153
DPDK	Attributed power	0.0039	0.239
USR	QoS score	0.0785	0.907
USR	Attributed power	0.3038	0.960

6. Experimental Setup

This section describes the workload data and evaluation protocol used to assess the controllers introduced in Section 4. Unless explicitly stated otherwise, all controller comparisons in this section are performed offline in the same digital twin under replayed workload conditions.

6.1. Workload Traces and Time Base

The primary evaluation workload is derived from the NetMob 2023 dataset [26], which provides spatially disaggregated, service-level mobile traffic demand across French cities. We use the downlink traffic volume attributed to Netflix streaming in the city of Lyon, aggregated across spatial units to obtain a single city-level time series. We refer to this as the *Main trace*. The raw traffic is normalized and rescaled so that the maximum offered load does not exceed 400 Mbps, keeping the peak demand within the profiled operating range and within the single-instance USR capacity boundary identified during profiling. This scaling avoids extrapolating the profiling models beyond the measurement envelope while preserving a workload that exercises both realization-mode selection and USR scale selection.

To assess cross-trace generalization, we derive a second workload from the same NetMob 2023 dataset using the downlink traffic volume attributed to YouTube streaming in the same city and time period. We refer to this as the *YouTube trace*. The same normalization and rescaling procedure is applied. Because the YouTube service class exhibits a different diurnal pattern and load distribution compared to Netflix, this trace provides an independent check of whether policies trained on the Main trace transfer to an unseen traffic profile without retraining.

The Main trace spans the period from 2019-04-01 00:00 to 2019-05-05 23:45, yielding $N = 3{,}359$ control slots at the 15-minute control period $\Delta = 15$ min. The dataset is split chronologically at a 75/25 ratio, giving $N_{\text{tr}} = 2{,}519$ training slots ($\approx$26 days) and $N_{\text{te}} = 840$ test slots ($\approx$9 days). No shuffling is applied in order to preserve temporal structure and avoid leakage across time. The YouTube trace follows the same temporal extent and split ratio.

One-step-ahead traffic forecasts $\hat{\rho}_{t+1}$, used by the forecast-augmented and hybrid observation schemas, are produced by an LSTM-based sequence model trained on a temporal split of the Main traffic series (no shuffling). The predictor uses calendar features and a 96-sample input window. Table 2 reports its held-out accuracy in project load-scaled traffic units.

Table 2
Held-out accuracy of the traffic predictor used by the forecast-augmented and hybrid observation schemas.

Predictor	Horizon	RMSE	R^2
LSTM-based traffic predictor	t+1	19.12	0.898

6.2. Key Configuration Parameters

Table 3 summarizes the main parameters used in the experimental campaign, including the cooldown length, reward weights, and PPO hyperparameters.

All PPO training is performed on a single CPU without GPU acceleration. At the target budget of 3×10^5 timesteps, training one feed-forward PPO seed takes approximately 7.4 minutes (443 s wall-clock), while one recurrent PPO seed takes approximately 11.6 minutes (693 s). The five-seed robustness runs reported in Section 7.9 require about 37 minutes total for the feed-forward variant and about 58 minutes for the recurrent variant.

Table 3
Main experimental parameters used in the evaluation.

Parameter	Value
Control period	$\Delta = 15$ min
Cooldown length	$I_c = 4$
QoS target threshold	$\tau = 0.9$
SEC scaling in reward	$\alpha = 100$
QoS penalty weight	$\lambda_{\text{QoS}} = 30$
Switch-to-DPDK cost c_{DPDK}	0.03
Per-instance USR launch cost c_{USR}	0.012
Per-instance scale-down cost c_{dn}	0.003
Discount factor	$\gamma = 0.995$
PPO rollout steps	$n_{\text{steps}} = 1024$
Batch size	64
Epochs per update	10
Learning rate	10^{-4}
Entropy coefficient	0.15
Clip range	0.2
GAE parameter	0.9
Total training steps	3×10^5
Train/test split ratio	0.75/0.25

7. Results & Analysis

This section reports the performance of the controllers introduced in Section 4 under the experimental protocol described in Section 6. All results are computed on the held-out test split, in offline replay through the digital twin, and under the same actuation guard from Section 3.4. Unless otherwise stated, comparisons are performed on the executed configuration sequence $\{c_t\}$ and the corresponding twin outcomes $\{(P_t, Q_t)\}$.

7.1. Evaluation Metrics

We quantify energy efficiency, QoS compliance, and orchestration behavior using three metric groups.

Energy is measured by integrating the interval-averaged attributed-power trace over the evaluation horizon:

$$E_{\text{tot}} = \sum_{t=0}^{N_{\text{te}}-1} P_t \Delta. \tag{13}$$

In addition, we report the average specific energy consumption

$$\overline{\text{SEC}} = \frac{1}{N_{\text{te}}} \sum_{t=0}^{N_{\text{te}}-1} \frac{P_t}{\max(\rho_t, \epsilon)}. \tag{14}$$

QoS compliance is summarized through the scalar outcome Q_t produced by the digital twin. Given the compliance target τ, we report the violation rate

$$\upsilon = \frac{1}{N_{\text{te}}} \sum_{t=0}^{N_{\text{te}}-1} \mathbb{I}\{Q_t < \tau\}, \tag{15}$$

and the average compliance indicator $\overline{Q} = \frac{1}{N_{\text{te}}} \sum_t Q_t$.

Orchestration behavior is characterized by disaggregating reconfigurations into two event types. Type-switch events, counted as

$$n_{\text{sw}} = \sum_{t=1}^{N_{\text{te}}-1} \mathbb{I}\{m_t \neq m_{t-1}\}, \tag{16}$$

capture mode changes between DPDK and USR. Horizontal scaling events, counted as

$$n_{\mathrm{sc}} = \sum_{t=1}^{N_{\mathrm{te}}-1} \mathbb{I}\{m_t = m_{t-1},\, k_t \neq k_{t-1}\}, \tag{17}$$

capture changes in the number of active USR instances under a fixed mode. We also report n_{bl}, the number of change requests absorbed by the cooldown guard without actuation.

7.2. Overall Comparison Across Controllers

Table 4 compares the controllers in terms of energy, QoS, and orchestration behavior. Since the static and rule-based baselines are deterministic, they are reported as single values. The PPO results are given as mean ± standard deviation over five independently trained seeds on the held-out evaluation split. The rule-based controller uses the same configuration space $\mathcal{C}$ and the same cooldown guard, so the differences reflect the decision policy rather than different actuation semantics.

Table 4
Overall results on the main held-out evaluation split. Lower is better for E_{tot}, $\overline{\mathrm{SEC}}$, and v. n_{sw} counts type-switch events (DPDK↔USR); n_{sc} counts horizontal scaling events (USR instance-count changes); n_{bl} counts change requests blocked by the cooldown guard. E_{tot} is in Wh and $\overline{\mathrm{SEC}}$ in Wh/Mbit. PPO rows report mean ± standard deviation over five seeds.

Controller	E_{tot}	$\overline{\mathrm{SEC}}$	v	n_{sw}	n_{sc}	n_{bl}
Offline-optimal reference	116.21	0.00681	0.0097	47	23	0
PPO (feed-forward)	**122.58 ± 1.50**	**0.00748 ± 0.00015**	**0.00333 ± 0.00362**	16.4 ± 3.5	10.6 ± 7.4	12.0 ± 9.5
PPO (recurrent)	125.29 ± 5.11	0.00804 ± 0.00124	0.00610 ± 0.00434	20.0 ± 10.2	5.8 ± 3.6	22.8 ± 19.8
Rule-based baseline	123.92	0.00775	0.0014	24	0	0
Static DPDK	147.59	0.01355	0.0000	-	-	-
Static USR($k = 1$)	178.36	0.00908	0.0014	-	-	-

The offline-optimal reference achieves the lowest attributed energy, but it does so with full knowledge of the future load trajectory and with a substantially more aggressive actuation sequence, including 47 type switches and a QoS violation rate of 0.97%. It is therefore best interpreted as a reference bound for the modeled problem rather than as a deployable policy.

Among the causal controllers, feed-forward PPO performs best on average. It achieves lower mean attributed energy, lower mean SEC, and a lower violation rate than recurrent PPO, while also showing tighter variance across seeds. Compared with the rule-based baseline, the reduction in total attributed energy is modest (122.58 vs. 123.92 Wh, about 1.1%), whereas the difference in average SEC is somewhat clearer (0.00748 vs. 0.00775 Wh/Mbit, about 3.5%). Feed-forward PPO also remains closer to the offline-optimal reference than the rule-based controller (5.48% vs. 6.64% above the planner), despite operating without future information.

The comparison with the rule-based baseline is important mainly because the gain is modest but consistent and does not come at the expense of QoS. The more interesting difference appears in the control behavior: the learned policy replaces some disruptive mode switches with horizontal scaling actions, whereas the rule-based controller does not exploit that option.

The recurrent PPO variant is less energy-efficient and less stable across seeds, with wider variance in every reported metric. Static USR($k = 1$) also performs poorly overall. Although the user-space realization is more load-proportional, a single instance operates too close to its saturation region and cannot scale out when demand increases.

7.3. Energy Efficiency

Figure 6 reports the interval-averaged attributed-power trace P_t over the evaluation horizon for the learning-based controller and representative baselines. The static DPDK baseline exhibits a nearly load-independent attributed-power profile in this process-attributed measurement setup. The static user-space baseline is more load-dependent, with attributed power increasing more visibly with offered load.

The learning-based controller reduces total attributed energy by adapting the executed configuration to the traffic regime. In low-to-moderate load periods, it often favors user-space configurations that reduce the attributed cost of keeping DPDK active continuously; in higher-load periods, it selects configurations that avoid QoS degradation

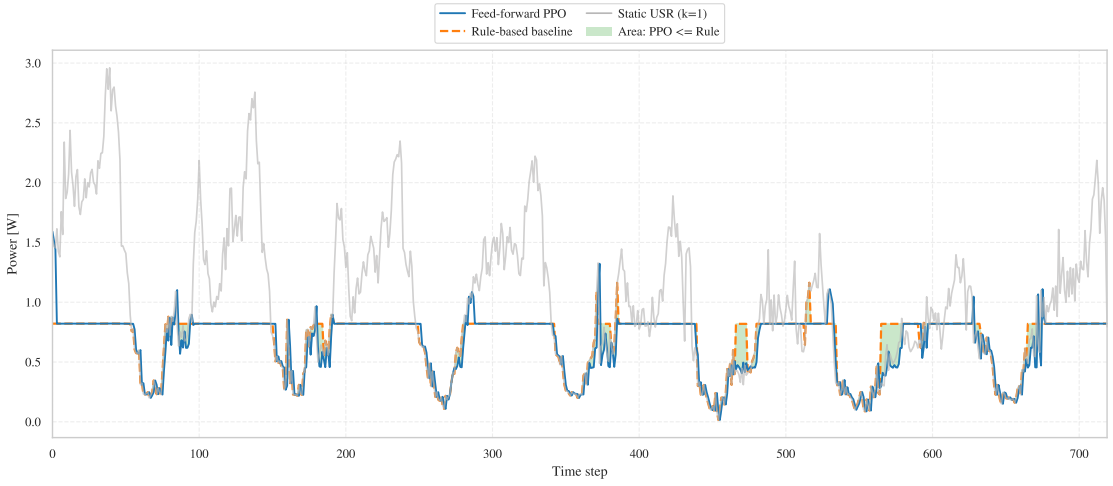


Figure 6: Interval-averaged attributed power P_t over the test horizon for the evaluated controllers.

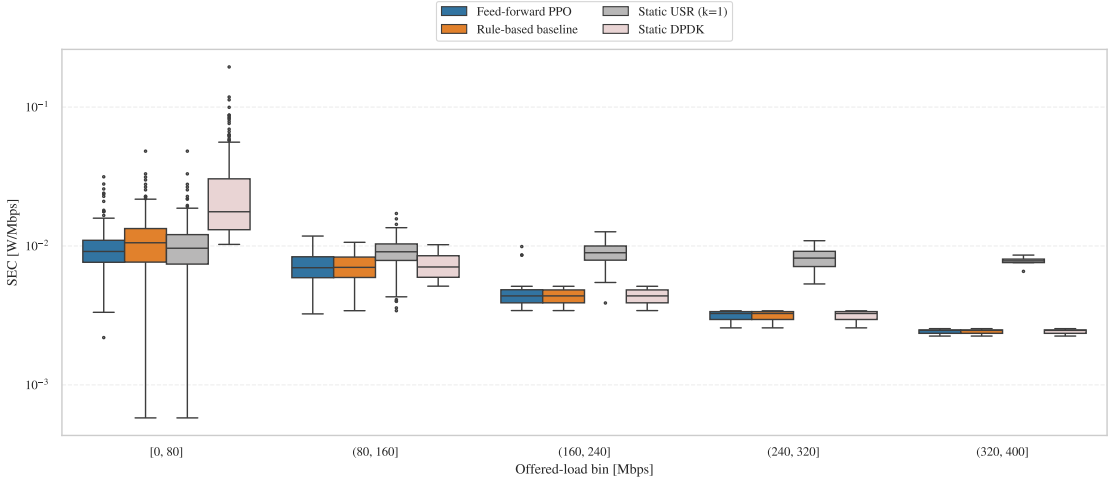


Figure 7: Specific energy consumption SEC_t by offered-load bins.

without incurring unnecessary energy cost. The resulting behavior improves the average energy-per-bit relative to static provisioning.

To clarify where savings are realized, Figure 7 reports SEC_t aggregated by offered-load bins. The feed-forward PPO controller achieves its clearest advantage in the intermediate regime, where the trade-off between keeping DPDK active and scaling the user-space realization is most pronounced. The improvement relative to the rule-based baseline is also distributed broadly across the trace: over 51% of time steps, the feed-forward PPO controller draws less attributed power than the rule-based controller. This suggests that the gain does not come from a single isolated regime.

7.4. QoS Compliance

QoS behavior is assessed through the compliance indicator Q_t and the violation rate υ in (15). Figure 8 shows the evolution of Q_t over time and highlights intervals below the target τ. Static user-space provisioning may violate the target during sustained high load when the fixed instance count becomes insufficient. Static DPDK generally improves compliance but at the cost of higher attributed energy over the full trace.

The feed-forward PPO controller maintains compliance by allocating sufficient capacity when traffic rises while avoiding persistent over-provisioning during low-load intervals. The violation rate reported in Table 4 confirms that

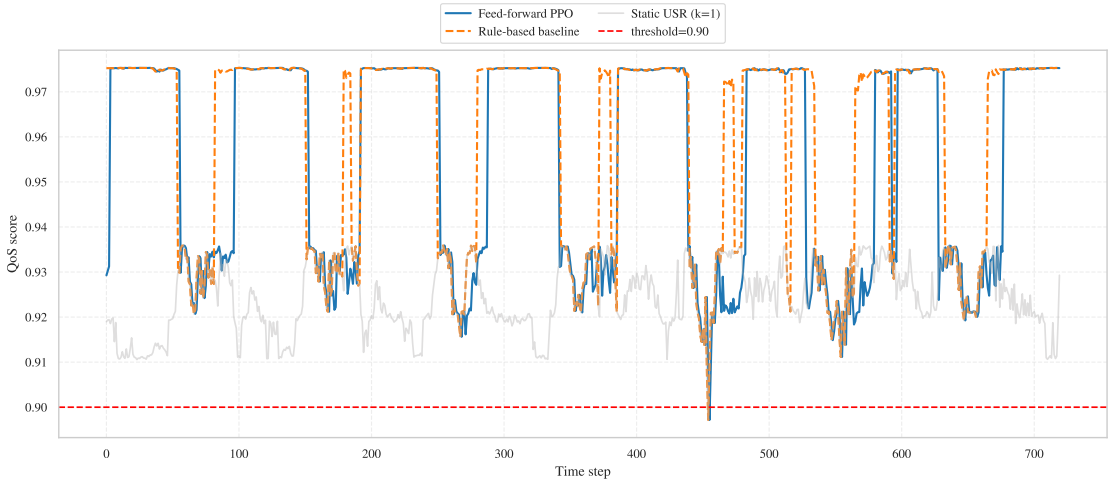


Figure 8: QoS-compliance indicator Q_t over the test horizon. The dashed line marks the target τ.

the energy gains are not achieved by systematically trading off QoS. When violations occur, they tend to coincide with sharp load transitions or with intervals in which the cooldown guard delays an immediate reconfiguration.

7.5. Orchestration Behavior and Stability

Figure 9 reports the executed configuration sequence $\{c_t\}$ over time together with the offered load. The sequence illustrates how the learned policy partitions the load range into regions associated with different UPF modes and scale levels. Compared with the reactive heuristic, the feed-forward PPO controller produces stable actuation patterns with limited oscillation around regime boundaries.

Table 4 disaggregates reconfigurations into type-switch events (n_{sw}) and horizontal scaling events (n_{sc}). This distinction is operationally relevant because a type switch is more disruptive than adding or removing a USR replica. The feed-forward PPO controller issues 17 type switches and 7 scaling events (24 total), while the rule-based baseline issues 24 type switches and no scaling events. The total number of actuations is therefore the same, but the learned controller substitutes some disruptive mode changes with lower-cost scaling operations. The recurrent PPO controller is the most conservative, issuing only 16 type switches and no scaling events, with 7 requests absorbed by the cooldown guard.

The configuration sequences in Figure 9 reveal qualitatively different strategies. During intermediate-load periods, the feed-forward PPO controller frequently selects the 2×USR configuration, exploiting horizontal scaling to serve moderate demand at lower attributed power than DPDK while avoiding single-instance USR saturation. In contrast, the rule-based baseline oscillates between DPDK and 1×USR in similar intervals because it cannot exploit the intermediate scaling level. The recurrent PPO controller tends to remain in DPDK for extended periods, forgoing the energy savings available at lower loads.

The rule-based baseline does not exploit horizontal scaling because the profiling-derived thresholds for mode switching and scale selection fall too close together: the DPDK→USR switching threshold is 90.47 Mbps and the 1×USR→2×USR scaling threshold is 87.13 Mbps, while the hysteresis dead-band width is approximately 23 Mbps. As a result, the dead-bands overlap and no traffic region triggers a scaling action without also triggering a mode switch. The recurrent PPO variant does not exploit horizontal scaling within the available training budget, as the entropy-collapse behavior documented in Section 7.7 causes the agent to commit to a conservative two-mode strategy before exploring the intermediate configuration.

7.6. Ablation on Observation Schemas and Policy Parameterization

This subsection isolates the impact of temporal context in the observation and in the policy parameterization. We compare the instantaneous schema against history-augmented, forecast-augmented, and hybrid schemas, and we compare feed-forward PPO against recurrent PPO under matched training budgets.

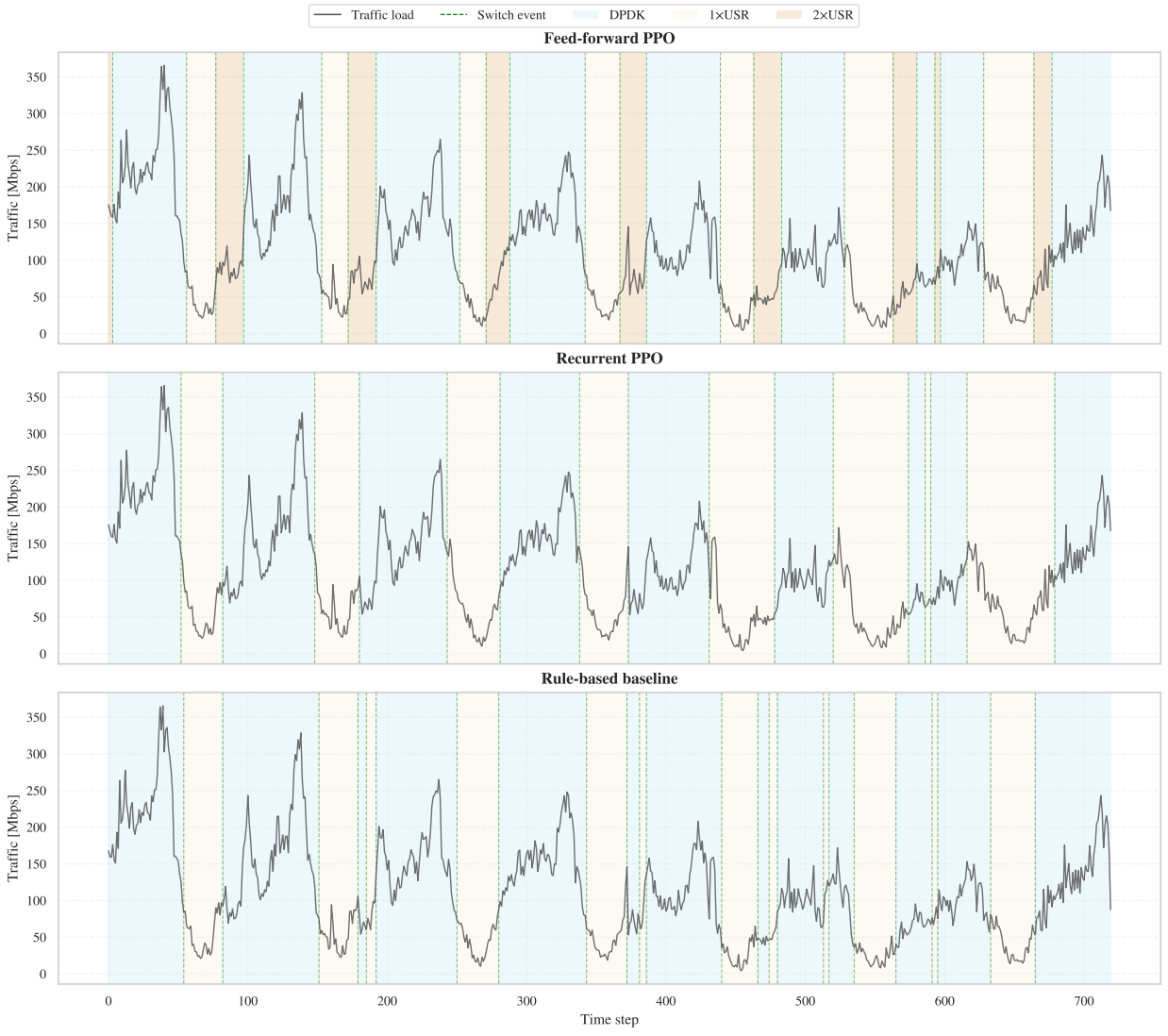


Figure 9: Executed configuration sequence $\{c_t\}$ over the test horizon.

Table 5 reports energy, QoS, and stability metrics for all considered combinations, disaggregated into type-switch (n_{sw}) and scaling (n_{sc}) events. For feed-forward PPO, only the hybrid schema activates horizontal scaling (7 scaling events), while all other schemas produce exclusively type switches. In our experiments, the combination of traffic history and forecast therefore provided the clearest context for exploiting the intermediate USR scaling level rather than oscillating between single-instance USR and DPDK. The instantaneous schema triggers the most type switches ($n_{\text{sw}} = 24$) despite comparable energy, indicating less stable actuation. The forecast-only schema is the most switch-conservative ($n_{\text{sw}} = 15$), but it yields slightly higher energy because it does not exploit scaling.

For recurrent PPO, the instantaneous and forecast-only schemas collapse to a fixed DPDK configuration ($n_{\text{sw}} = 0$, $E_{\text{tot}} = 147.79$ Wh), matching the static DPDK baseline. This suggests that, within the available training budget, those recurrent settings fail to learn a useful switching strategy; the underlying cause(entropy collapse) is examined in Section 7.7. The history-augmented schema unlocks effective switching ($n_{\text{sw}} = 16$, $E_{\text{tot}} = 125.74$ Wh) with no scaling events, indicating a more conservative two-mode strategy. The hybrid recurrent schema produces only 4 type switches with no scaling, but occupies a weaker energy position ($E_{\text{tot}} = 132.38$ Wh). Overall, feed-forward PPO is

more robust to schema choice and is the only controller that clearly exploits the full configuration space in the reported experiments.

Table 5
Ablation results for observation schemas and policy parameterization. n_{sw} = type-switch events; n_{sc} = scaling events. Bold entries indicate the best result per policy variant. † denotes that the recurrent policy collapsed to a fixed configuration (no switching).

Policy	Observation schema	E_{tot} (Wh)	v	n_{sw}	n_{sc}
PPO (feed-forward)	instantaneous	121.03	0.0014	24	0
PPO (feed-forward)	history (W)	122.21	0.0014	19	0
PPO (feed-forward)	forecast (H)	123.45	0.0014	15	0
PPO (feed-forward)	**hybrid**	**120.79**	**0.0014**	**17**	**7**
PPO (recurrent)	instantaneous	147.79	0.0000	$0^{\dagger}$	0
PPO (recurrent)	forecast (H)	147.79	0.0000	$0^{\dagger}$	0
PPO (recurrent)	**history (W)**	**125.74**	**0.0014**	**16**	**0**
PPO (recurrent)	hybrid	132.38	0.0042	4	0

7.7. Training Convergence Diagnostics

To understand the training dynamics behind the ablation and seed-robustness results, we examine learning curves, policy entropy evolution, and multi-seed convergence behavior. Figure 10 presents these diagnostics across all observation schemas and both policy parameterizations.

Panel (a) shows evaluation return over training. For feed-forward PPO, all four observation schemas converge to high evaluation returns within approximately 30k training steps, after which the policy collapses sharply (returns drop to approximately -975). This late-stage collapse does not affect deployment: best-model checkpointing selects the pre-collapse policy for evaluation. For recurrent PPO, only the history-augmented schema converges to a competitive return; the instantaneous and forecast-only schemas fail to learn a useful switching policy, consistent with their collapse to static DPDK in Table 5.

Panel (b) reveals the underlying cause. The instantaneous and forecast-only recurrent schemas exhibit entropy collapse, with policy entropy falling below 0.5 within the first 50k steps. This premature commitment to a single action prevents meaningful exploration of the switching strategy and explains the fixed-configuration behavior observed in the ablation. In contrast, the history-augmented recurrent schema and all feed-forward schemas maintain sufficient entropy throughout training, enabling effective policy learning.

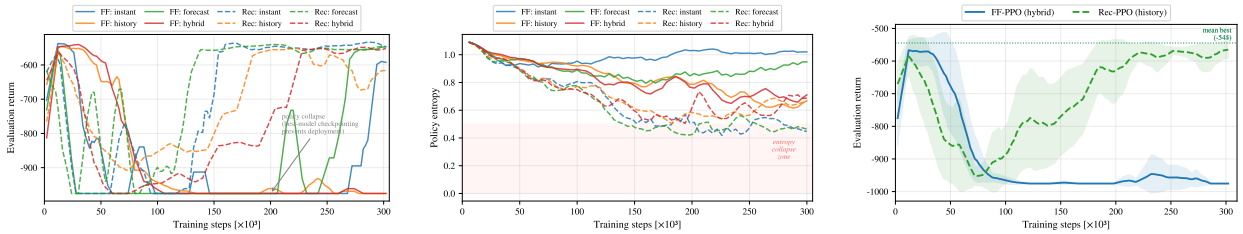
Panel (c) shows convergence across five independently seeded training runs for the two best-performing configurations identified in the ablation: feed-forward PPO with the hybrid observation schema and recurrent PPO with the history schema. Both controllers reach comparable peak returns (mean best return approximately -545), but feed-forward PPO converges faster and exhibits tighter seed-to-seed variation, consistent with the aggregate statistics in Section 7.9. The shaded bands indicate ± 1 standard deviation across seeds.

7.8. Sensitivity Analysis

This subsection examines the sensitivity of the feed-forward PPO controller to three design parameters: the cooldown length I_c, the QoS compliance target τ, and the maximum number of admissible user-space instances k_{max}.

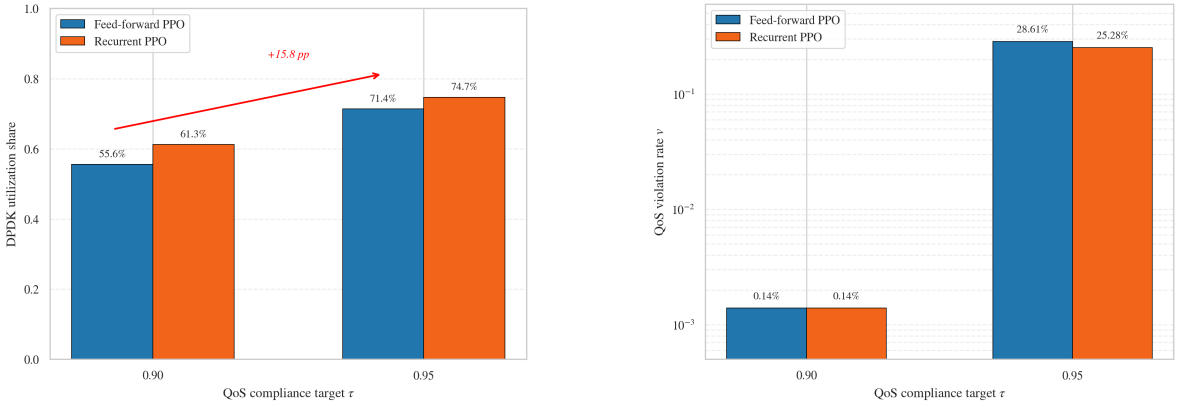
7.8.1. Cooldown Length I_c

We analyze sensitivity to the cooldown length I_c using the feed-forward PPO controller with the hybrid observation schema. With $I_c = 0$ (no cooldown), the controller produces 36 reconfigurations and a violation rate of $v = 0.0097$, indicating that unrestricted actuation leads to oscillatory behavior that degrades QoS. Increasing the cooldown to $I_c = 2$ already suppresses most oscillations ($n_{reconf} = 17$, $v = 0.0014$) with only a marginal energy penalty. The default setting $I_c = 4$ maintains the same violation rate and reconfiguration count while achieving the lowest total attributed energy at 120.80 Wh. Further increasing to $I_c = 8$ slightly raises energy (121.55 Wh) and violation rate ($v = 0.0042$), as the extended lockout prevents timely adaptation to rapid load changes. These results suggest that the chosen cooldown of $I_c = 4$ is a reasonable operating point for this testbed-derived setting.



(a) Evaluation return. (b) Policy entropy. (c) Multi-seed robustness.

Figure 10: Training convergence diagnostics. (a) Evaluation return over training for all observation schemas and both policy parameterizations (single seed). The arrow highlights late-stage policy collapse in feed-forward PPO, which does not affect deployment due to best-model checkpointing. (b) Policy entropy evolution. Recurrent PPO with instantaneous and forecast-only schemas exhibits entropy collapse below the shaded zone, explaining the fixed-configuration behavior in Table 5. (c) Evaluation return across five independent seeds for the two best-performing configurations, with ±1 standard deviation bands.



(a) DPDK utilization share. (b) QoS violation rate.

Figure 11: Effect of QoS-compliance target τ on DPDK utilization share and QoS violation rate for feed-forward and recurrent PPO.

7.8.2. QoS Compliance Target τ

The QoS-compliance target τ controls the threshold below which the reward penalty is applied and directly shapes how conservatively the agent selects configurations. Figure 11 reports the effect of varying $\tau \in \{0.90, 0.95\}$ on the DPDK utilization share and QoS violation rate for both PPO variants.

At $\tau = 0.95$, the feed-forward PPO controller allocates DPDK in 71.4% of time steps, a shift of +15.8 percentage points relative to the $\tau = 0.90$ operating point (55.6%). The recurrent variant exhibits a similar shift, increasing its DPDK share from 61.3% to 74.7% (+13.4 pp). Despite this more conservative behavior, the violation rate rises sharply to $\upsilon = 0.20$–0.37 for both variants, indicating that the available USR configurations cannot consistently satisfy the stricter target under this workload. The number of reconfigurations also drops at $\tau = 0.95$, as the agents remain in DPDK more often. In this setting, $\tau = 0.90$ is therefore better matched to the QoS capabilities of the current profiled configurations.

7.8.3. Maximum User-Space Instance Count $k_{\max}$

The maximum number of admissible user-space instances $k_{\max}$ determines the scaling headroom available to the agent. Table 6 reports the effect of restricting $k_{\max} \in \{1, 2\}$ at $\tau = 0.90$ for both PPO variants.

The feed-forward PPO controller benefits from the larger configuration space: expanding from $k_{\max} = 1$ to $k_{\max} = 2$ reduces total attributed energy by 2.0% (from 123.32 to 120.80 Wh) and reduces the reconfiguration count from 20 to 17, because the two-instance USR level provides an intermediate option for moderate-to-high loads. In contrast, the

Table 6
Effect of maximum instance count $k_{\max}$ on energy and stability at $\tau = 0.90$.

Policy	$k_{\max}$	E_{tot} (Wh)	v	n_{reconf}
PPO (feed-forward)	1	123.32	0.0014	20
PPO (feed-forward)	2	120.80	0.0014	17
PPO (recurrent)	1	121.80	0.0014	14
PPO (recurrent)	2	125.74	0.0014	16

recurrent PPO variant achieves its lowest energy (121.80 Wh) and fewest reconfigurations (14) at $k_{\max} = 1$, suggesting that it does not effectively exploit the additional scaling level within the available training budget.

7.9. Seed Robustness

To assess training stability, we evaluate both PPO variants across five independently seeded training runs using fixed hyperparameters and identical evaluation conditions. Since the main comparison reports mean ± standard deviation across seeds, Table 7 is retained as a detailed per-seed breakdown that exposes the spread and the main outliers behind those aggregate values.

Table 7
Per-seed results for feed-forward and recurrent PPO across five training seeds on the main held-out evaluation split.

Policy	Seed	E_{tot} (Wh)	v	n_{reconf}
PPO (feed-forward)	42	122.94	0.0028	18
PPO (feed-forward)	123	122.33	0.0014	14
PPO (feed-forward)	**456**	**120.79**	**0.0014**	**17**
PPO (feed-forward)	789	121.98	0.0097	20
PPO (feed-forward)	1024	124.88	0.0014	24
PPO (recurrent)	42	122.11	0.0083	38
PPO (recurrent)	123	134.01	0.0014	18
PPO (recurrent)	**456**	**125.74**	**0.0014**	**16**
PPO (recurrent)	789	122.72	0.0097	14
PPO (recurrent)	1024	121.92	0.0097	14

The feed-forward PPO controller exhibits relatively tight seed-to-seed variation, with total attributed energy ranging from 120.79 to 124.88 Wh and an aggregate result of 122.58 ± 1.50 Wh. Its average SEC is likewise stable at 0.00748 ± 0.00015 Wh/Mbit, and its mean violation rate remains low at 0.00333 ± 0.00362. The recurrent PPO controller shows much wider variation, from 121.92 to 134.01 Wh, with a substantially broader aggregate spread of 125.29 ± 5.11 Wh and 0.00804 ± 0.00124 Wh/Mbit in average SEC. It also exhibits stronger behavioral variability, including a high-switching outlier (seed 42) and substantially more blocked requests on average. These results confirm that the feed-forward controller is the more reliable choice for this setting.

7.10. Cross-Trace Generalization

The results presented so far are obtained on the Main trace, which is also the trace used for training. To assess whether the learned policies generalize to an unseen traffic profile, we evaluate all controllers on the YouTube trace described in Section 6.1. The PPO policies are not retrained; the same five-seed checkpoints trained on the Main trace are replayed on the YouTube workload through the same digital twin.

Table 8 reports the results. The static baselines confirm that the YouTube trace exercises a different load distribution: Static USR($k = 1$) consumes less energy (165.33 Wh vs. 178.76 Wh on Main), and the rule-based baseline achieves zero QoS violations, indicating that the YouTube load stays further from the USR saturation boundary.

The feed-forward PPO controller reduces mean total attributed energy by 15.8% relative to Static DPDK on the YouTube trace (124.47 vs. 147.78 Wh), which is comparable to the 16.9% reduction observed on the Main trace. However, its margin over the rule-based baseline narrows: mean energy is slightly higher (+0.73%), while mean SEC remains lower (0.00782 vs. 0.00797 Wh/Mbit, −1.9%). This narrower margin is expected because the YouTube trace is

Table 8
Overall results on the YouTube trace (cross-trace generalization). The PPO policies are identical to those trained on the Main trace and are not retrained. Notation follows Table 4.

Controller	E_{tot}	$\overline{\text{SEC}}$	v	n_{sw}	n_{sc}	n_{bl}
Offline-optimal reference	116.70	0.00698	0.0166	31	18	0
PPO (feed-forward)	**124.47 ± 1.60**	**0.00782 ± 0.00025**	**0.00442 ± 0.00721**	16.4 ± 3.5	10.8 ± 6.8	14.8 ± 13.7
PPO (recurrent)	126.77 ± 5.74	0.00868 ± 0.00230	0.00996 ± 0.00909	20.4 ± 13.0	4.4 ± 3.2	25.6 ± 25.6
Rule-based baseline	123.57	0.00797	0.0000	16	0	1
Static DPDK	147.78	0.01521	0.0000	-	-	-
Static USR($k = 1$)	165.33	0.00917	0.0000	-	-	-

less demanding and the rule-based controller already achieves zero violations, leaving less room for an adaptive policy to exploit intermediate configurations.

The feed-forward PPO controller continues to outperform the recurrent variant on YouTube, with lower mean energy (-1.8%), lower mean SEC (-9.9%), and lower mean violation rate. The recurrent variant also exhibits wider seed-to-seed variation, with one outlier seed (seed 123) reaching 136.38 Wh. These patterns are consistent with the Main-trace findings and confirm that the relative ranking of the two PPO parameterizations is preserved across workloads.

Overall, the feed-forward PPO controller demonstrates that the energy-efficiency gains over static provisioning are robust to workload change, while the margin over the engineered rule-based baseline is workload-dependent and narrows on a less demanding trace.

7.11. Statistical Significance Analysis

The multi-seed evaluation in Section 7.9 reports mean and standard deviation across five independently trained seeds. To assess whether the observed differences are statistically robust, we apply the Wilcoxon signed-rank test [27] to the paired per-seed outcomes on the Main trace. This non-parametric test is appropriate for small paired samples and does not assume normality.

We test the central comparison of the paper: feed-forward PPO versus the rule-based baseline. For each of the five seeds, we compute the paired difference in total energy E_{tot}, mean SEC $\overline{\text{SEC}}$, and QoS violation rate v. The null hypothesis is that the median paired difference is zero (two-tailed).

Table 9 reports the test statistics. With $n = 5$ paired observations, the minimum achievable p-value for the Wilcoxon signed-rank test is $p = 0.0625$, which cannot reach the conventional $\alpha = 0.05$ significance level regardless of how consistent the results are. This is a structural consequence of the sample size rather than an indication of weak effects.

Table 9
Wilcoxon signed-rank test results for feed-forward PPO vs. rule-based baseline on the Main trace. n_{better}: seeds where FF-PPO improves over the baseline; W: test statistic; p: two-tailed p-value. The minimum achievable p with $n = 5$ is 0.0625.

Metric	n_{better} / n	Δ mean	W	p
E_{tot} (Wh)	4/5	-1.74 (-1.40%)	1.0	0.1250
$\overline{\text{SEC}}$ (Wh/Mbit)	5/5	-0.00030 (-3.86%)	0.0	0.0625
v	0/5	$+0.00194$	0.0	0.5000

The SEC comparison yields $W = 0.0$ and $p = 0.0625$: all five seeds achieve lower SEC than the rule-based baseline, and the test statistic reaches the minimum achievable p-value. This is the strongest possible result obtainable with five paired samples. The total energy comparison yields $p = 0.1250$, with four of five seeds improving over the baseline. The QoS violation rate shows no significant difference ($p = 0.5000$), consistent with the design intent that the PPO controller should match, not necessarily improve, the baseline's QoS behavior.

Figure 12 visualizes the paired per-seed differences and the Wilcoxon test outcome for the feed-forward PPO versus rule-based comparison.

These results confirm that the SEC improvement is consistent across all five seeds, and the inability to reach $\alpha = 0.05$ is a limitation of the sample size rather than of the effect magnitude. Increasing the number of independently trained seeds in future work would allow conventional significance testing.

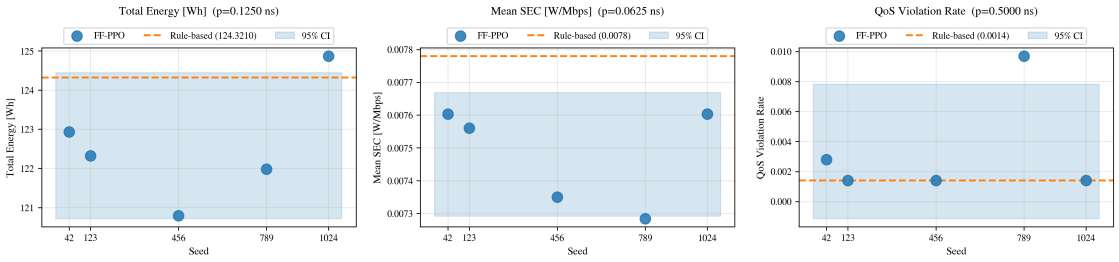


Figure 12: Wilcoxon signed-rank test visualization for feed-forward PPO vs. rule-based baseline on the Main trace. Each point represents one training seed; the dashed line marks zero difference.

8. Conclusion

This work framed UPF orchestration in the 5G Core as a sequential decision problem under practical deployment constraints, including cooldown guards, asymmetric reconfiguration costs, and the need to balance energy efficiency against QoS. A PPO-based controller was trained offline in a measurement-grounded digital twin and evaluated on both held-out traffic and a second, unseen workload.

Across five training seeds, the feed-forward PPO controller reduced total attributed energy to 122.58 ± 1.50 Wh, corresponding to a 16.9% improvement over static DPDK provisioning and a 1.1% reduction compared to a carefully tuned rule-based baseline, while maintaining a low QoS violation rate (0.00333 ± 0.00362). On the YouTube trace, the reduction over static DPDK remained at 15.8% without retraining, indicating that the learned policy transfers across workloads with different temporal characteristics. Although the gain over the rule-based controller is modest, it is consistent across seeds, with all runs achieving lower SEC. The corresponding Wilcoxon signed-rank test reaches $p = 0.0625$, which reflects the limited sample size rather than the absence of a systematic effect.

More importantly, the learned policy exhibits a different control behavior. Instead of relying primarily on mode switching, it makes use of horizontal scaling to handle intermediate load regimes, reducing the need for disruptive transitions. This option is effectively unavailable to the threshold-based controller, where overlapping dead-bands prevent scaling from being selected independently.

The proposed framework is compatible with the NWDAF/MDAF analytics architecture defined in 5G systems and is designed to extend naturally to larger configuration spaces, including multiple UPF realizations and scalable instance deployments. In the present work, evaluation is conducted in a deterministic digital twin, which captures the mean system behavior but does not reproduce transient effects or runtime variability. The most immediate next step is deployment in a live closed-loop setting on the physical testbed. This requires completion of the runtime UPF reconfiguration mechanism and its integration with the relevant control-plane procedures needed to support service continuity during reconfiguration.

A. Notation Reference

Table 10 summarizes the main symbols used throughout the paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used Claude (Anthropic) for improving language clarity, LaTeX formatting assistance, and editorial proofreading. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Data availability

The evaluation workload is derived from the publicly available NetMob 2023 dataset. Data for the profiling campaign, code, and trained models will be made available upon reasonable request.

Table 10
Summary of notation.

Symbol	Description
$\mathcal{M}$	Set of UPF realization modes
$\mathcal{K}_m$	Admissible instance counts for mode m
$c_t = (m_t, k_t)$	Executed configuration at slot t
$\mathcal{C}$	Configuration space
a_t	Configuration command issued by the agent
Δ	Control period (slot duration)
x_t	Observation vector at slot t
ρ_t	Offered load at slot t
P_t	Interval-averaged UPF-attributed power
Q_t	Scalar QoS-compliance score
E_{tot}	Total attributed energy over the evaluation
SEC_t	Specific energy consumption at slot t
u_t	Cooldown counter (slots since last reconfiguration)
I_c	Cooldown length (in slots)
τ	QoS-compliance target threshold
υ	QoS violation rate
π_θ	Parameterized policy
r_t	Reward at slot t
γ	Discount factor
α	SEC scaling weight in reward
λ_{QoS}	QoS penalty weight in reward
c_{DPDK}, c_{USR}, c_{dn}	Switch-to-DPDK, per-instance USR launch, scale-down costs
$\mathcal{T}(\cdot)$	Digital-twin transition function
$p_m(\rho)$, $q_m(\rho)$	Profiling models for power and QoS
n_{sw}	Number of type-switch events
n_{sc}	Number of horizontal scaling events
n_{bl}	Number of cooldown-blocked requests

Acknowledgements

This work was supported in part by the European Union's Horizon Europe programme under the SUNRISE-6G project (Grant Agreement No. 101139075) and in part by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, Mission 4, Component 2, Investment 1.3, CUP F83C22001690001, partnership on "Telecommunications of the Future" (PE00000001 – program "RESTART").

References

[1] A. Bellin, F. Granelli, D. Munaretto, A measurement-based approach to analyze the power consumption of the softwarized 5G core, Computer Networks 244 (2024) 110312. doi:10.1016/j.comnet.2024.110312.

[2] S. Chen, J. Chen, H. Li, Joint optimization of UPF placement and traffic routing for 5G core network user plane, Computer Communications 216 (2024) 1–15. doi:10.1016/j.comcom.2023.12.029.

[3] C. Freitag, M. Berners-Lee, K. Widdicks, B. Knowles, G. S. Blair, A. Friday, The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations, Patterns 2 (9) (2021) 100340. doi:10.1016/j.patter.2021.100340.

[4] P. Soto, D. D. Vleeschauwer, M. Camelo, Y. D. Bock, K. D. Schepper, C.-Y. Chang, P. Hellinckx, J. F. Botero, S. Latré, Towards autonomous VNF auto-scaling using deep reinforcement learning, in: Proc. IEEE Int. Conf. Smart Data Services (SDS), 2021. doi:10.1109/SDS54264.2021.9731854.

[5] T. Tournaire, H. Castel-Taleb, E. Hyon, T. Hoché, Generating optimal thresholds in a hysteresis queue: Application to a cloud model, in: Proc. IEEE 27th Int. Symp. Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), 2019, pp. 283–294. doi:10.1109/MASCOTS.2019.00040.

[6] J. Pei, P. Hong, M. Pan, J. Liu, J. Zhou, Optimal VNF placement via deep reinforcement learning in SDN/NFV-enabled networks, IEEE J. Selected Areas in Communications 38 (2) (2020) 263–278. doi:10.1109/JSAC.2019.2959181.

[7] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, D. I. Kim, Applications of deep reinforcement learning in communications and networking: A survey, IEEE Communications Surveys & Tutorials 21 (4) (2019) 3133–3174. doi:10.1109/COMST.2019.2916583.

[8] A. Bellin, M. Centenaro, F. Granelli, A preliminary study on the power consumption of virtualized edge 5G core networks, in: Proc. IEEE 9th Int. Conf. Network Softwarization (NetSoft), 2023, pp. 420–425. doi:10.1109/NetSoft57336.2023.10175489.

[9] S. Christakis, T. Tsourdinis, N. Makris, T. Korakis, S. Fdida, Evaluation of user plane function implementations in real-world 5G networks, in: Proc. IEEE INFOCOM Workshops (CNERT), 2024, pp. 1–6. doi:10.1109/INFOCOMWKSHPS61880.2024.10620666.

[10] J. Rischke, C. Vielhaus, P. Sossalla, J. Wang, F. H. P. Fitzek, Comparison of UPF acceleration technologies and their tail-latency for URLLC, in: Proc. IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN), 2022, pp. 19–25. doi:10.1109/NFV-SDN56302.2022.9974798.

[11] R. Bolla, P. Ghiorzo, R. Bruschi, L. Ivaldi, F. Davoli, C. Lombardo, B. Siccardi, Comparison of the performance and energy efficiency evaluation of 5G user-plane functions, in: Proc. IEEE GreenCom, 2024, pp. 361–367. doi:10.1109/iThings-GreenCom-CPSCom-SmartData-Cybermatics62450.2024.00076.

[12] S. A. Borji, R. Bruschi, C. E. Costa, C. Lombardo, Dynamic energy-efficient user plane function selection in 5G networks (2024). doi:https://doi.org/10.5281/zenodo.18484420.

[13] C. Pham, N. H. Tran, S. Ren, W. Saad, C. S. Hong, Traffic-aware and energy-efficient vNF placement for service chaining: Joint sampling and matching approach, IEEE Trans. Services Computing 13 (1) (2020) 172–185. doi:10.1109/TSC.2017.2671867.

[14] R. Lin, L. He, S. Luo, M. Zukerman, Energy-aware service function chaining embedding in NFV networks, IEEE Trans. Services Computing 16 (2) (2023) 1158–1171. doi:10.1109/TSC.2022.3162328.

[15] A. Gausseran, F. Giroire, B. Jaumard, J. Moulierac, Be scalable and rescue my slices during reconfiguration, in: Proc. IEEE Int. Conf. Communications (ICC), 2020, pp. 1–6. doi:10.1109/ICC40277.2020.9148871.

[16] R. Solozabal, J. Ceberio, A. Sanchoyerto, L. Zabala, B. Blanco, F. Liberal, Virtual network function placement optimization with deep reinforcement learning, IEEE J. Selected Areas in Communications 38 (2) (2020) 292–303. doi:10.1109/JSAC.2019.2959183.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).

[18] Z. Chen, B. Yin, H. Zhu, Y. Li, M. Tao, W. Zhang, Mobile communications, computing, and caching resources allocation for diverse services via multi-objective proximal policy optimization, IEEE Trans. Communications 70 (7) (2022) 4498–4512. doi:10.1109/TCOMM.2022.3173005.

[19] Z. Kuai, G. Luo, J. Wang, T. Lin, Y. Cheng, Y. Xu, Fair virtual network function mapping and scheduling using proximal policy optimization, IEEE Trans. Communications 71 (1) (2023) 471–485. doi:10.1109/TCOMM.2022.3211071.

[20] G. L. Santos, P. T. Endo, T. Lynn, D. Sadok, J. Kelner, A reinforcement learning-based approach for availability-aware service function chain placement in large-scale networks, Future Generation Computer Systems 136 (2022) 93–109. doi:10.1016/j.future.2022.05.021.

[21] Z. Tao, W. Xu, X. You, Digital twin-accelerated online deep reinforcement learning for admission control in sliced communication networks, IEEE Trans. Communications 73 (4) (2025) 2490–2504. doi:10.1109/TCOMM.2024.3476430.

[22] H. Wang, Y. Wu, G. Min, W. Miao, A graph neural network-based digital twin for network slicing management, IEEE Trans. Industrial Informatics 18 (2) (2022) 1367–1376. doi:10.1109/TII.2020.3047843.

[23] R. Bolla, C. Lombardo, P. Bono, N. S. Martinelli, R. Bruschi, B. Siccardi, An open-source prototype of network data analytics function for next-generation 5/6g environments, in: 2023 IEEE Globecom Workshops (GC Wkshps), 2023, pp. 720–725. doi:10.1109/GCWkshps58843.2023.10465014.

[24] M. Akbari, R. Bolla, R. Bruschi, C. Lombardo, N. S. Martinelli, B. Siccardi, A management data analytics function for ethical 6G networks, Computer Networks 270 (2025) 111487. doi:10.1016/j.comnet.2025.111487.

[25] 3GPP, Technical specification 23.501: System architecture for the 5G system (5GS), Tech. rep., 3rd Generation Partnership Project, release 17 (2022).

[26] NetMob, NetMob 2023 dataset: Mobile traffic demand records, available: https://netmob.org (2023).

[27] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83. doi:10.2307/3001968.